

MailSense: Actionable Email Detection Using NLP

Binary Classification of Email Text into Actionable and Non-Actionable

Project Report

Submitted by

Md Ariful Alam Mahim (Roll: 2107023)

Md Jubair Husain (Roll: 2107029)

September 2026

1 Introduction

Email inboxes mix content that requires the recipient to do something, such as requests, proposals and questions, with content that does not, such as newsletters, notifications and purely informative statements. An overlooked request can lead to a missed deadline or an unanswered question, so automatically flagging actionable content is a useful aid for email triage. The task is closely related to earlier work on classifying email into speech acts such as request and propose [2].

MailSense treats this as a supervised binary text-classification problem and compares three approaches: (A) TF-IDF features with Logistic Regression, (B) a BiLSTM initialized with pretrained Word2Vec embeddings, and (C) a fine-tuned pretrained BERT model (bert-base-uncased). All three models use the same cleaned data, the same train/validation/test split and the same evaluation code, so that differences in the results reflect the modelling approach rather than differences in data handling.

2 Problem Statement and Objectives

2.1 Problem Statement

Given a piece of email text, the system must predict one of two labels: **Actionable (Yes)**, if the text asks for or proposes an action by the recipient, or **Non-Actionable (No)** otherwise. Actionable is the positive class. A false negative is therefore an actionable text predicted as non-actionable, i.e. an item the user could miss; for this reason recall and the number of false negatives are reported alongside the other metrics. Because the dataset consists of individual email sentences or short snippets rather than complete messages, the task is more precisely sentence-level classification of actionable email content.

2.2 Objectives

- Prepare a cleaned, de-duplicated dataset and a single stratified train/validation/test split shared by all models.
- Implement three classifiers: TF-IDF + Logistic Regression, a BiLSTM initialized with pretrained Word2Vec embeddings, and fine-tuned BERT.
- Evaluate all models on the same held-out test set using accuracy, precision, recall, F1-score, ROC-AUC and confusion matrices, with Actionable as the positive class.
- Compare the results, examine the error types of each model and state the limitations of the study.

3 Dataset and Preprocessing

3.1 Dataset Description

We use the training file Ask0729-fixed.txt of the Parakwee Labs Email Intent Dataset [1]. Each line holds a Yes/No label and one sentence taken from the Enron email corpus [3]; the dataset's positive label corresponds primarily to the request and propose speech acts of [2]. The file contains **3,657 examples: 1,719 Actionable (47.0%) and 1,938 Non-Actionable (53.0%)**, so the classes are close to balanced. All lines were well-formed (0 rows rejected at parsing).

3.2 Data Cleaning and Preparation

A shared cleaning function is applied to every example: Unicode (NFKC) normalization, decoding of HTML entities, replacement of URLs, email addresses and numbers with the placeholder tokens \$LINK, \$EMAIL and \$NUM, removal of HTML tags, and whitespace normalization. Stop words and punctuation are kept, because words such as please, could and you, and the question mark, are strong cues for requests. After cleaning, examples that were empty, had conflicting labels, or were exact duplicates were removed. Only the duplicate filter removed anything: 6 examples, several of which became identical only after email addresses were replaced by \$EMAIL. The **final dataset contains 3,651 examples** (1,718 Actionable, 1,933 Non-Actionable).

3.3 Train–Validation–Test Split

The final dataset is divided into 70% training, 15% validation and 15% test data by stratified random splitting (seed 42), which preserves the class ratio in each part. The split is saved once and reused unchanged by all three models; the test set (548 examples) is used only for the final evaluation. Table 1 summarizes the data at each stage.

Table 1. Dataset size and class distribution at each stage.

Stage	Actionable (Yes)	Non-Actionable (No)	Total
Raw dataset	1,719	1,938	3,657
After cleaning	1,718	1,933	3,651
Training set (70%)	1,202	1,353	2,555
Validation set (15%)	258	290	548
Test set (15%)	258	290	548

4 Methodology

4.1 TF-IDF + Logistic Regression

The baseline represents each text as a sparse TF-IDF vector [4] of lower-cased word unigrams and bigrams, with sub-linear term-frequency scaling and n-grams kept only if they occur in at

least 2 and at most 90% of training documents. The vectorizer is fitted on the training split only, giving 6,572 features. A Logistic Regression classifier with L2 regularization ($C = 1.0$), balanced class weights and the liblinear solver (maximum 2,000 iterations) is trained on these vectors [9].

4.2 Word2Vec + BiLSTM

Model B is a BiLSTM initialized with pretrained Word2Vec embeddings. A vocabulary of 2,450 entries (words with frequency of at least 2 in the training split, plus padding and unknown tokens) is built, and its 300-dimensional embedding matrix is initialized from the pretrained Google News Word2Vec vectors [5]; 2,338 of 2,448 vocabulary words (95.51%) were found, and the remaining words were initialized randomly. The embeddings are trainable and are fine-tuned together with the rest of the network. A single-layer bidirectional LSTM [6, 7] with 128 hidden units per direction reads the sequence (maximum 64 tokens); the final forward and backward states are concatenated and passed through dropout ($p = 0.3$) and a linear output layer. The model has 1,175,834 trainable parameters.

4.3 Pretrained BERT

Model C fine-tunes bert-base-uncased [8], a 12-layer Transformer encoder that produces contextual token representations. It is loaded through the Hugging Face Transformers library [11] with a two-class sequence-classification head (a linear layer on the [CLS] representation). Inputs are WordPiece-tokenized and truncated or padded to 64 tokens, and all 109,483,778 parameters are fine-tuned.

4.4 Experimental Setup

The experiments were run in a single Kaggle notebook on an NVIDIA Tesla T4 GPU using scikit-learn [9], PyTorch [10], Transformers [11] and gensim [12]. The random seed is fixed to 42 for all models. The TF-IDF model is fitted once on the training set with the fixed configuration above. The two neural models minimize cross-entropy loss; after every epoch they are evaluated on the validation set, the checkpoint with the highest validation F1-score is kept, and training stops early if the validation F1 does not improve for a set number of epochs (patience). The selected checkpoint is then evaluated once on the test set. No hyperparameter search was performed; the settings in Table 2 were used as fixed.

Table 2. Training settings of the neural models.

Setting	Word2Vec + BiLSTM	BERT
Optimizer	Adam [14]	AdamW [13]
Learning rate	1×10^{-3} , constant	2×10^{-5} , linear decay, 10% warm-up
Weight decay	0	0.01
Batch size	32	16
Maximum epochs / patience	20 / 4	4 / 2
Gradient-norm clipping	5.0	1.0
Maximum sequence length	64 tokens	64 WordPiece tokens

5 Evaluation Metrics

All models are evaluated by the same code on the same test set of 548 examples (258 Actionable, 290 Non-Actionable). A text is predicted Actionable when the model's predicted probability of the Actionable class is at least **0.5**. With Actionable as the positive class, TP, TN, FP and FN denote true positives, true negatives, false positives and false negatives.

- **Accuracy** = $(TP + TN) / (TP + TN + FP + FN)$: the proportion of all texts classified correctly.
- **Precision** = $TP / (TP + FP)$: the proportion of texts flagged as actionable that are actionable.
- **Recall** = $TP / (TP + FN)$: the proportion of actionable texts that are found.
- **F1-score** = $2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$: the harmonic mean of precision and recall; also the model-selection criterion on the validation set.
- **ROC-AUC**: the area under the ROC curve, computed from the predicted probabilities. It measures how well actionable texts are ranked above non-actionable ones across all thresholds, independently of the 0.5 threshold.
- **Confusion matrix**: the counts of TP, TN, FP and FN, with true labels as rows and predicted labels as columns.

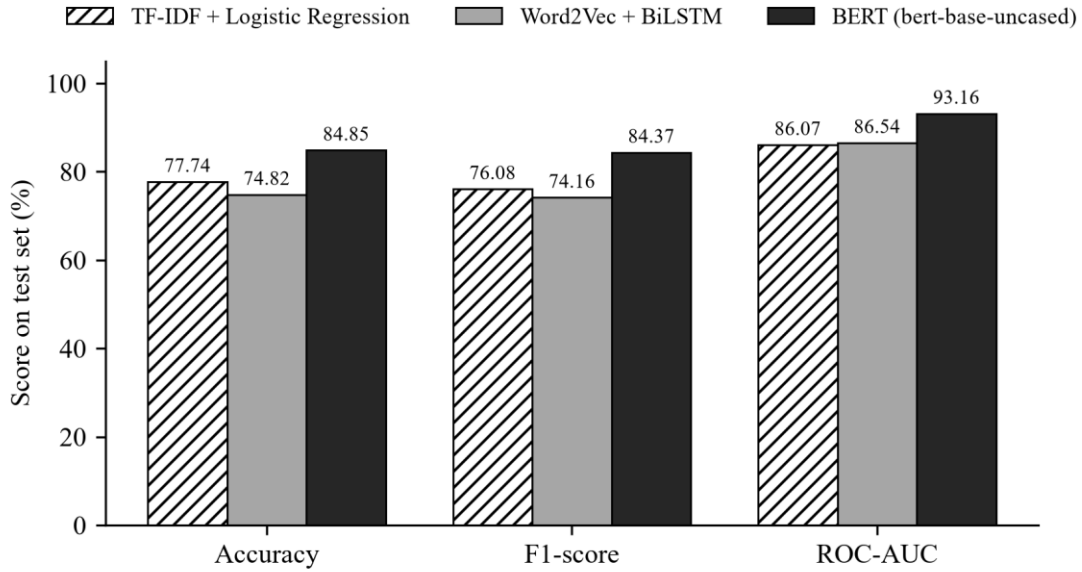
6 Results and Analysis

6.1 Overall Model Performance

Table 3 reports the test-set results of the single final run, taken from the recorded result files. Figure 1 compares accuracy, F1-score and ROC-AUC.

Table 3. Test-set results (n = 548; positive class = Actionable; threshold = 0.5).

Model	Accuracy	Precision	Recall	F1	ROC-AUC
TF-IDF + Logistic Regression	77.74%	76.98%	75.19%	76.08%	86.07%
Word2Vec + BiLSTM	74.82%	71.74%	76.74%	74.16%	86.54%
BERT (bert-base-uncased)	84.85%	82.05%	86.82%	84.37%	93.16%

**Figure 1.** Accuracy, F1-score and ROC-AUC of the three models on the test set.

BERT achieved the highest measured value on all five metrics among the three models in this experiment, exceeding the TF-IDF baseline by 7.11 percentage points in accuracy, 8.29 points in F1-score and 7.09 points in ROC-AUC. The TF-IDF baseline scored higher than the BiLSTM on accuracy, precision and F1-score, whereas the BiLSTM had higher recall (76.74% vs. 75.19%) and a slightly higher ROC-AUC (86.54% vs. 86.07%).

6.2 Confusion Matrix Analysis

Figure 2 shows the test-set confusion matrices. Of the 258 actionable test examples, BERT missed 34 (false negatives), compared with 64 for TF-IDF + Logistic Regression and 60 for the BiLSTM. BERT also produced the fewest false positives (49). The BiLSTM produced the most false positives (78, against 58 for the baseline), which explains its lower precision despite its slightly higher recall.

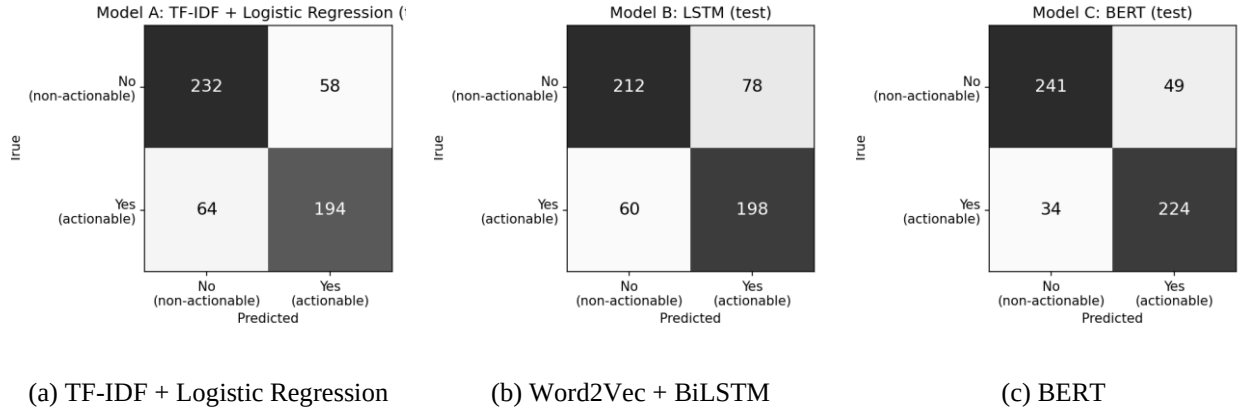


Figure 2. Test-set confusion matrices (rows: true label; columns: predicted label).

6.3 Training Performance

Figures 3 and 4 plot the training and validation loss recorded for the two neural models. For the BiLSTM, the best validation F1-score (81.75%) was reached at epoch 3; it did not improve over the next four epochs, so training stopped after epoch 7 and the epoch-3 checkpoint was used. The training loss kept falling (to 0.078) while the validation loss rose after epoch 3 (to 0.650), indicating overfitting.

For BERT, the validation F1-score improved in every epoch, so all 4 epochs were run and the final checkpoint was selected (validation F1 87.38%). Its validation loss was lowest after epoch 1 and then increased although validation accuracy and F1 improved, which suggests the model became more confident on the examples it still misclassified. Checkpoints were selected by validation F1, not by validation loss.

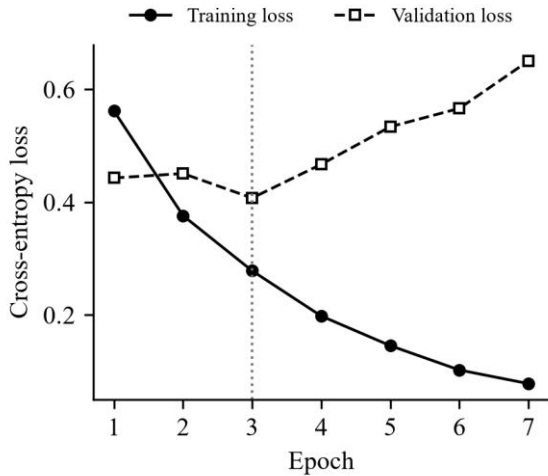


Figure 3. BiLSTM training loss vs. validation loss.

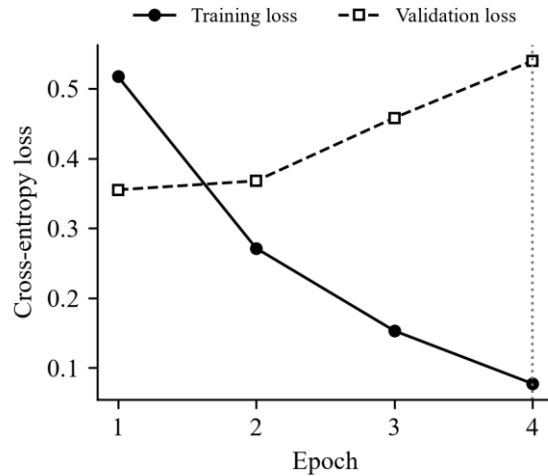


Figure 4. BERT training loss vs. validation loss.

The dotted vertical line marks the checkpoint selected by validation F1-score.

6.4 Discussion

Contextual pretraining. Whether a sentence is actionable often depends on its form and on how words combine, not only on individual words: the same verb appears in "Could you send the file?" and "I sent the file." BERT represents each token in the context of the whole sentence and brings knowledge from large-scale pretraining, which plausibly explains its higher scores here. This comes at a much higher cost: about 109.5 million fine-tuned parameters and roughly 30 seconds per epoch on the T4 GPU, against about one second per epoch for the BiLSTM.

A strong lexical baseline. Actionable sentences often contain explicit cues such as please, let me know or can you, which unigram and bigram features capture directly. With only 2,555 training examples, a regularized linear model is less prone to overfitting than a network with over a million parameters, which is consistent with TF-IDF + Logistic Regression outperforming the BiLSTM on test accuracy and F1-score.

Generalization of the BiLSTM. The BiLSTM reached a validation F1-score of 81.75% but a test F1-score of 74.16%, the largest drop of the three models. Together with the rising validation loss, this suggests that its selected checkpoint fitted the particular validation sample and generalized less well. Its errors were skewed towards false positives. Its ROC-AUC was nevertheless slightly above the baseline's, so the two models rank examples comparably well overall and part of the difference in thresholded metrics depends on how their probabilities fall relative to the fixed 0.5 threshold.

7 Limitations

- **Single train/validation/test split.** All results come from one stratified split; performance on other splits was not measured, and no cross-validation was performed.
- **Limited dataset size.** The final dataset has 3,651 examples and the test set 548, so small differences, such as the 0.47-point ROC-AUC difference between the BiLSTM and the baseline, should not be treated as conclusive.
- **Sentence-level data.** The examples are individual sentences or snippets, not complete emails; the models do not use the rest of the message, the subject line, the sender or the thread.
- **Fixed threshold.** All models used a decision threshold of 0.5. A threshold chosen to favour recall could reduce missed actionable items at the cost of more false positives; this was not explored.
- **Single experimental run.** Each model was trained once with one random seed and one fixed configuration, without a hyperparameter search. Variability across seeds was not measured and no statistical significance tests were performed.

8 Conclusion

MailSense compared three NLP approaches for detecting actionable email content under a controlled protocol: 3,651 cleaned examples from the Parakweet Labs Email Intent Dataset, one stratified 70/15/15 split shared by all models, and the same evaluation code. On the 548-example test set, fine-tuned BERT achieved the highest measured performance among the three models in this experiment, with 84.85% accuracy, 84.37% F1-score and 93.16% ROC-AUC, and it missed the fewest actionable examples (34 of 258). TF-IDF + Logistic Regression (77.74% accuracy, 76.08% F1) remained a competitive low-cost baseline, while the BiLSTM initialized with pretrained Word2Vec embeddings (74.82% accuracy, 74.16% F1) showed higher recall but lower precision than the baseline and signs of overfitting.

Natural next steps, following from the limitations, are to repeat the experiments over several seeds and splits with significance testing, to tune the decision threshold for higher recall, and to extend the task from single sentences to complete email messages.

References

- [1] Parakweet Labs, Inc., “EmailIntentDataSet: Labeled training and test data for email intent machine learning (for sentence-level speech acts),” GitHub repository, 2014.
<https://github.com/ParakweetLabs/EmailIntentDataSet>
- [2] W. W. Cohen, V. R. Carvalho, and T. M. Mitchell, “Learning to classify email into ‘speech acts’,” in Proc. EMNLP, 2004, pp. 309–316.
- [3] B. Klimt and Y. Yang, “The Enron corpus: A new dataset for email classification research,” in Proc. ECML, 2004, pp. 217–226.
- [4] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [5] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, “Distributed representations of words and phrases and their compositionality,” in *Advances in NIPS*, vol. 26, 2013.
- [6] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [7] M. Schuster and K. K. Paliwal, “Bidirectional recurrent neural networks,” *IEEE Trans. Signal Processing*, vol. 45, no. 11, pp. 2673–2681, 1997.
- [8] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in Proc. NAACL-HLT, 2019, pp. 4171–4186.
- [9] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *JMLR*, vol. 12, pp. 2825–2830, 2011.
- [10] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in NeurIPS*, vol. 32, 2019.
- [11] T. Wolf et al., “Transformers: State-of-the-art natural language processing,” in Proc. EMNLP 2020: System Demonstrations, pp. 38–45.
- [12] R. Řehůřek and P. Sojka, “Software framework for topic modelling with large corpora,” in Proc. LREC 2010 Workshop on New Challenges for NLP Frameworks, pp. 45–50.
- [13] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in Proc. ICLR, 2019.
- [14] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in Proc. ICLR, 2015.